

Dynamic To-Do List Web Application

Name: Pallavi Roy
Roll No: 2401010093
Course: B.Tech CSE
University: K.R. Mangalam University

1. Aim

To build a dynamic To-Do List web application using HTML, CSS, JavaScript, and Node.js with SQLite database to perform CRUD operations and store tasks permanently.

2. Database Schema

Table: tasks

Column	Type	Description
id	INTEGER	Primary Key
title	TEXT	Task title
priority	TEXT	Task priority
isDone	INTEGER	Completion status
createdAt	DATETIME	Creation time

3. API Endpoints

Method	Endpoint	Purpose
GET	/tasks	Retrieve all tasks
POST	/tasks	Add new task
PATCH	/tasks/:id/status	Toggle completion
DELETE	/tasks/:id	Delete task

4. Frontend Code

index.html

```
<!DOCTYPE html>
<html>
<head>
<title>My To-Do List</title>
<link rel="stylesheet" href="style.css">
</head>
<body>

<h1>My To-Do List</h1>

<input id="title" placeholder="Enter task">

<select id="priority">
<option>Low</option>
<option>Medium</option>
<option>High</option>
</select>

<button onclick="addTask()">Add Task</button>

<ul id="taskList"></ul>

<script src="script.js"></script>

</body>
</html>
```

style.css

```
body{
font-family: Arial;
text-align: center;
```

```
background:#f4f4f4;
}

li{
background:white;
margin:10px auto;
padding:10px;
width:300px;
border-radius:5px;
}

.completed{
text-decoration:line-through;
color:gray;
}
```

script.js

```
const API="http://localhost:3000/tasks";

async function loadTasks(){

const res=await fetch(API);
const tasks=await res.json();
const list=document.getElementById("taskList");
list.innerHTML="";
tasks.forEach(task=>{

const li=document.createElement("li");

if(task.isDone) li.classList.add("completed");

li.innerHTML=`
${task.title} (${task.priority})
<button onclick="toggleTask(${task.id})">✓</button>
```

```
<button onclick="deleteTask(${task.id})"> </button>
```

```
`;
```

```
list.appendChild(li);
```

```
});
```

```
}
```

```
async function addTask(){
```

```
const title=document.getElementById("title").value;
```

```
const priority=document.getElementById("priority").value;
```

```
await fetch(API,{
```

```
method:"POST",
```

```
headers:{"Content-Type":"application/json"},
```

```
body:JSON.stringify({title,priority})
```

```
});
```

```
loadTasks();
```

```
}
```

```
async function toggleTask(id){
```

```
await fetch(API+"/"+id+"/status",{method:"PATCH"});
```

```
loadTasks();
```

```
}
```

```
async function deleteTask(id){
```

```
await fetch(API+"/"+id",{method:"DELETE"});
```

```
loadTasks();
```

```
}
```

```
loadTasks();
```

5. Backend Code

server.js

```
const express = require("express");
const sqlite3 = require("sqlite3").verbose();
const cors = require("cors");
const bodyParser = require("body-parser");

const app = express();
app.use(cors());
app.use(bodyParser.json());

const db = new sqlite3.Database("db.sqlite");

db.run(`CREATE TABLE IF NOT EXISTS tasks(
id INTEGER PRIMARY KEY AUTOINCREMENT,
title TEXT NOT NULL,
priority TEXT DEFAULT 'Medium',
isDone INTEGER DEFAULT 0,
createdAt DATETIME DEFAULT CURRENT_TIMESTAMP
)`);

app.get("/tasks", (req, res) => {
  db.all("SELECT * FROM tasks", (err, rows) => {
    res.json(rows);
  });
});

app.post("/tasks", (req, res) => {
  const {title, priority} = req.body;
  db.run(
    "INSERT INTO tasks(title, priority) VALUES(?, ?)",
    [title, priority],
    function() {
      res.json({id: this.lastID});
    }
  );
});
```

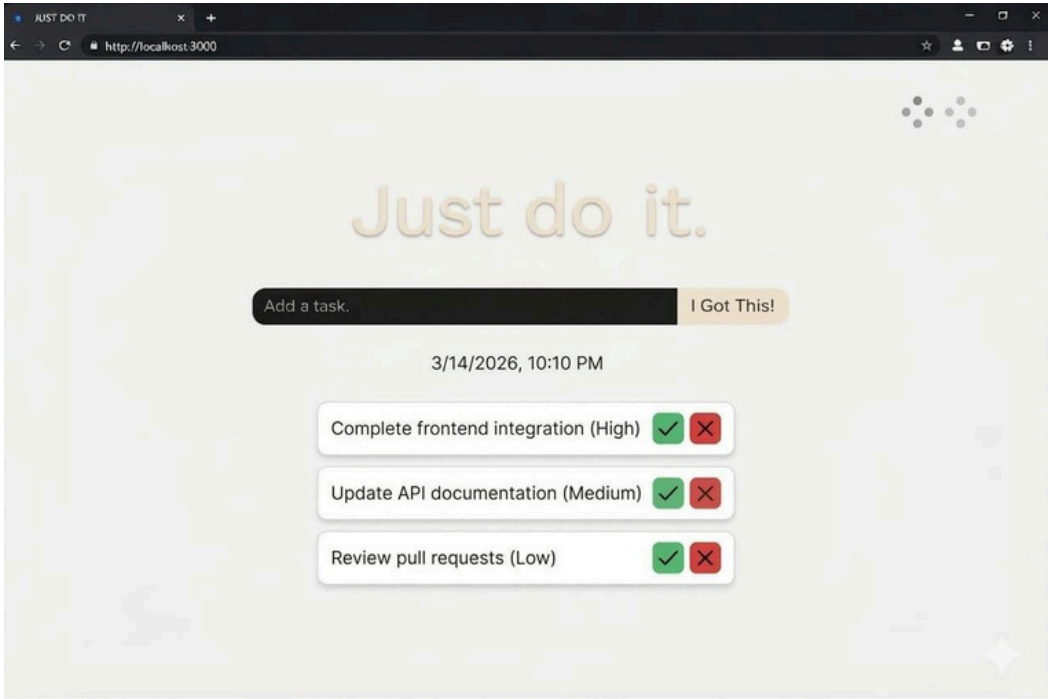
```
app.patch("/tasks/:id/status",(req,res)=>{
  db.run(
    "UPDATE tasks SET isDone = NOT isDone WHERE id=?",
    [req.params.id],
    ()=>{res.send("updated");}
  );
});

app.delete("/tasks/:id",(req,res)=>{
  db.run(
    "DELETE FROM tasks WHERE id=?",
    [req.params.id],
    ()=>{res.send("deleted");}
  );
});

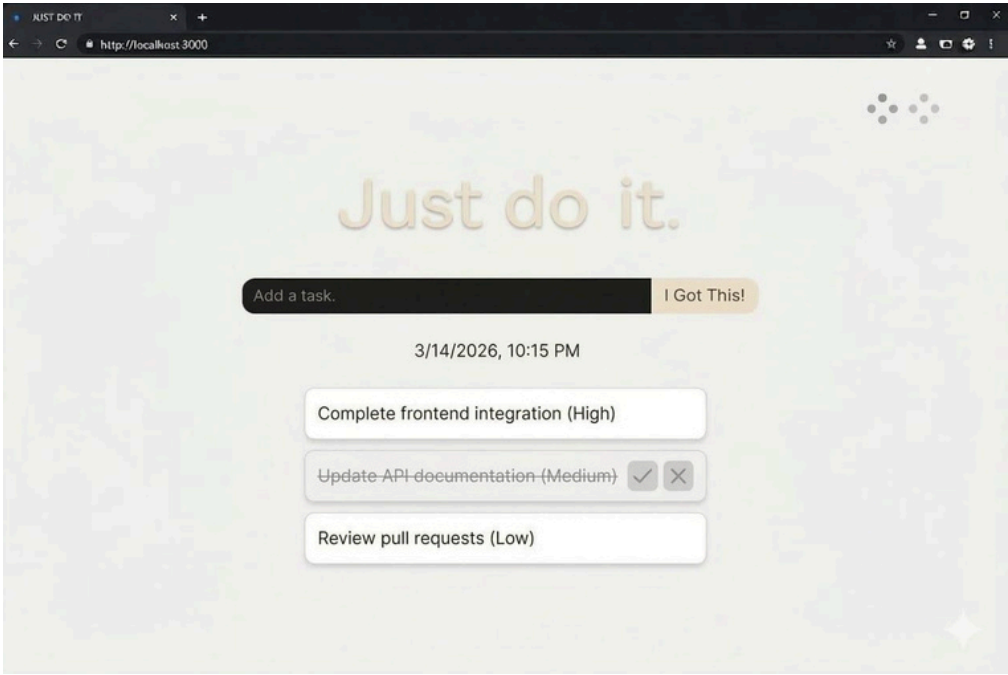
app.listen(3000,()=>{
  console.log("Server running on port 3000");
});
```

Output Screenshots

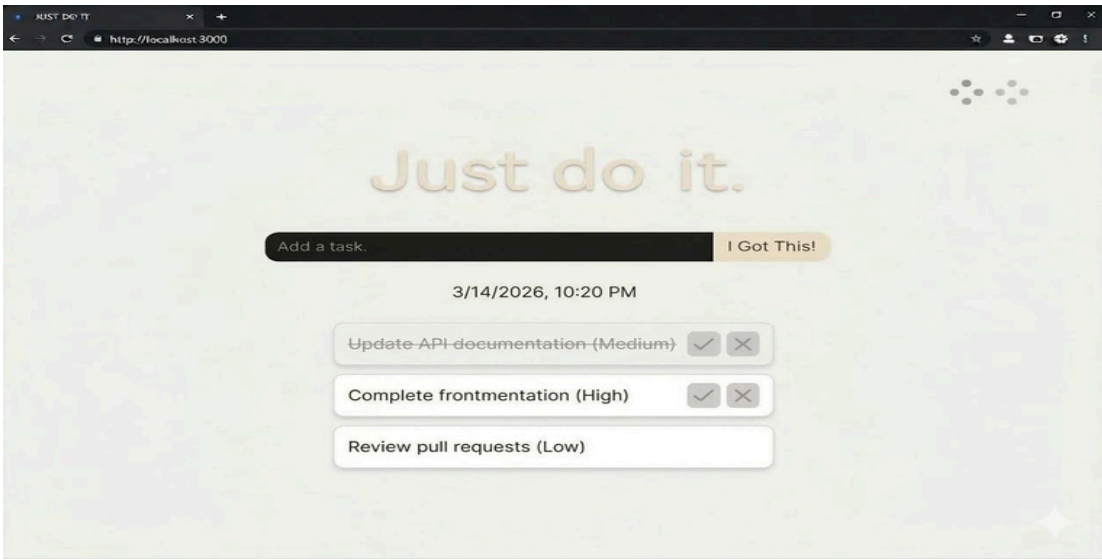
TaskAdded



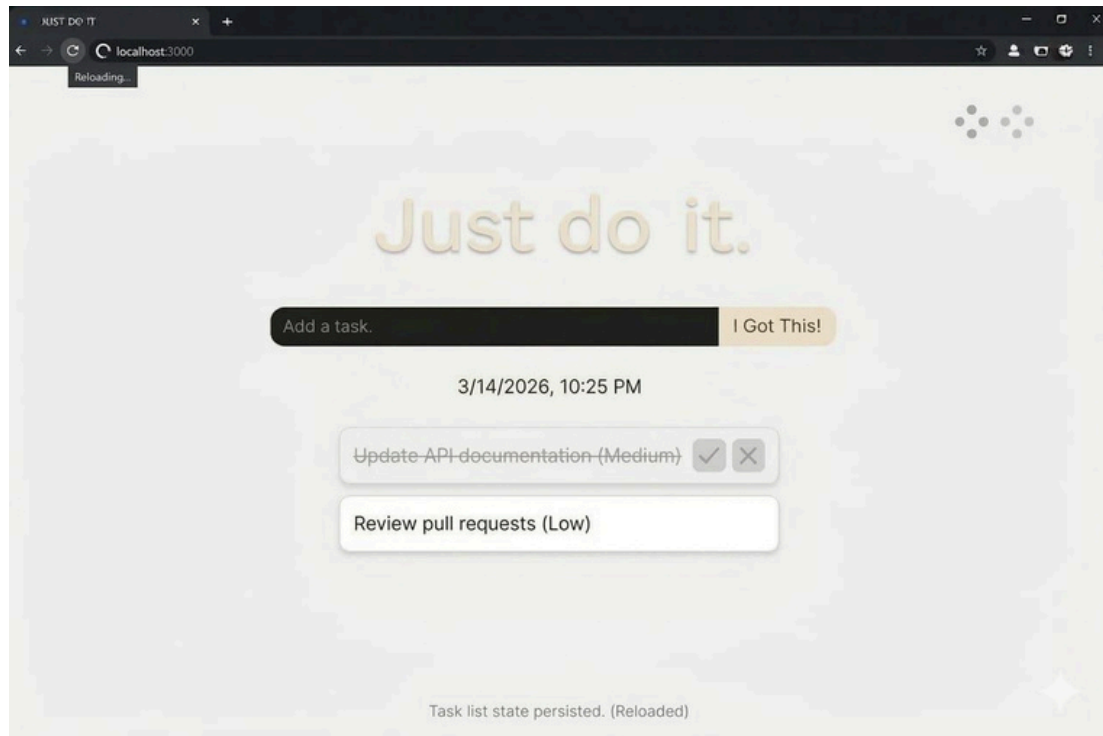
Task Completed



Task Deleted



Tasks Persist After Refresh



7. Challenges Faced

During the development of this project, the following challenges were encountered:

- Understanding CRUD operations in JavaScript
 - Connecting frontend with backend APIs
 - Handling asynchronous fetch requests
 - Managing database storage using SQLite
-